Sharper Boundaries: Position-Aware CRF Emissions and an Embedding Adapter for Predicting Proteolytic Peptides and Propeptides

Anonymous Author(s)
Affiliation
Address
email

Abstract

Proteolysis cuts precursor proteins into mature peptides and propeptides; predicting where is a sequence segmentation task with direct applications in proteomics, vaccine design, and disease research. The strongest existing approach couples frozen protein language model (pLM) embeddings with a CNN–BiLSTM encoder and an extended-state linear-chain conditional random field (CRF) decoder, but its CRF emissions are identical for every state of a given segment type at a given position, even though the decoder's own state structure already distinguishes a segment's start, interior, and end. We close this gap with two small, independent additions: a zero-initialized boundary head that adds position-specific corrections to the relevant CRF emissions, and a lightweight adapter that re-projects the pLM embedding before the encoder. Evaluated with the same 5×4 nested cross-validation DeepPeptide itself uses, on a rebuilt UniProtKB/Swiss-Prot 2026 dataset, both additions outperform the base architecture on ESM-2 embeddings individually ($+0.024$ and $+0.029$ F1) and combine to a $+0.058$ F1 gain over the 0.573 ± 0.025 base, slightly more than the sum of the two. On higher-capacity ESM-C 6B embeddings the base architecture is statistically indistinguishable from its ESM-2 counterpart under the same protocol (0.588 ± 0.016); the additions on that embedding, together with a larger set of exploratory ideas from earlier development, are reported in the appendix.

1 Introduction

Proteomic databases record intact sequences, not what proteases leave behind, so predicting the products of cleavage has direct value in several applied fields. In vaccine design the same prediction is useful at the design stage, to anticipate which fragments a multi-epitope construct will yield in vivo and to flag segments prone to unwanted cleavage. In disease research, proteolysis is disrupted in neurodegenerative, oncological and endocrine conditions, and a model of it lets expected and observed cleavage patterns be compared between healthy and diseased samples.

Where existing methods fall short. Most predictors of proteolytic cleavage are specific to one enzyme or one class of organism (Section 2), whereas what matters in practice is the combined effect of every protease active in an organism. The strongest general-purpose alternative, DeepPeptide [Teufel et al., 2023b], poses the task as full sequence segmentation on top of frozen pLM embeddings and an extended-state linear-chain CRF decoder, and remains, to our knowledge, the closest match to this formulation. But its CRF emissions are identical for every state of a given segment type at a given position, even though the decoder's own state graph already distinguishes a segment's start, interior, and end (a segment's path always moves start $\rightarrow$ interior $\rightarrow$ end) – the position-specific evidence a well-designed decoder could condition on is simply never computed by the encoder that feeds it.

Submitted to the AI for Drug Discovery workshop (NeurIPS 2026).

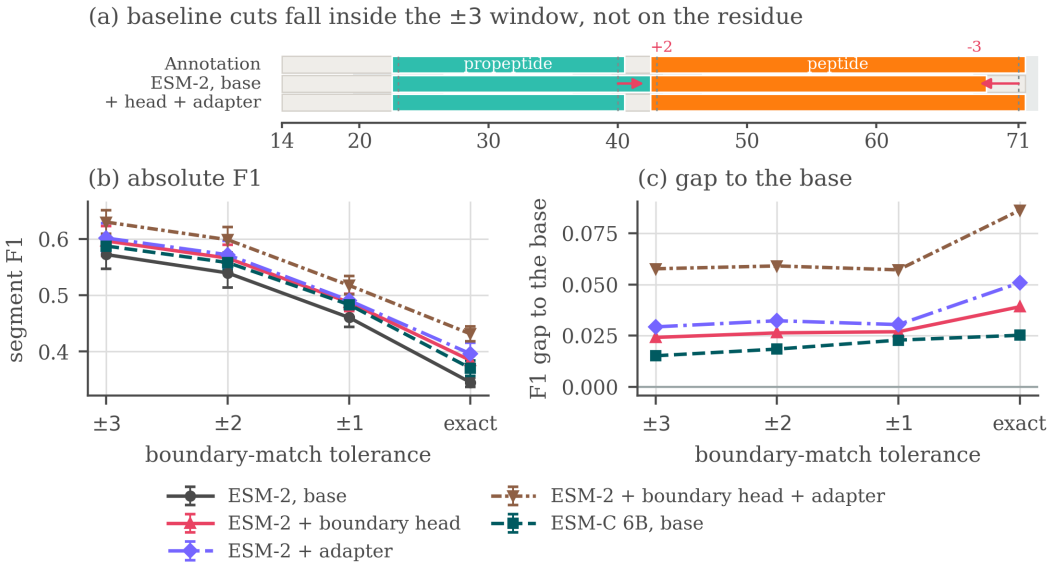

Figure 1: Boundary localization. (a) One precursor from the held-out fold of a nested-CV cell (drawn from residue 14 to its C-terminus), with the ±3 acceptance window shaded around each annotated cleavage site and the baseline's displacement in residues marked above it. Both of the baseline's cuts fall inside that window, so at the headline tolerance its segments count as found, while at an exact match they do not; the model with both additions places them on the annotated residue. (b) Segment F1 as the tolerance tightens from ±3 to an exact match. (c) The absolute gap to the ESM-2 base at each tolerance; a flat line would mean the curve is a parallel translation of the baseline, and none of them is flat. Corrected matcher throughout; error bars are the standard deviation across the five outer folds (Section 4.3).

What we do about it. We add two small, independent blocks to DeepPeptide's pLM + CNN–BiLSTM + CRF pipeline (Section 3): a boundary head that scores each position as a segment start, interior or end and adds those scores to the corresponding CRF emissions, and an adapter that re-projects the frozen pLM embedding before the encoder, since that embedding was trained for masked-residue recovery rather than cleavage-site localization. The head is zero-initialized, so training starts exactly at the baseline.

Both additions are evaluated under 5×4 nested cross-validation on the DeepPeptide dataset rebuilt from the 2026 UniProtKB/Swiss-Prot release (Sections 4.1 and 4.3); Appendix D records the single-split protocol used during development and the instability that retired it.

Our contributions are:

- A zero-initialized **boundary head** that supplies the CRF with the position-specific emissions its own state graph already distinguishes, at a cost of 4,678 parameters.
- A lightweight **input adapter** that re-projects frozen pLM embeddings before the encoder, evaluated alone and jointly with the head.
- Gains of $+0.024$, $+0.029$ and $+0.058$ F1 under nested cross-validation, decomposed into their parts: the head buys precision, the adapter buys recall, and both place segment boundaries more precisely, measured on the segments a variant and the base model both find.
- A quantified account of **why this benchmark resists a cleaner protocol**: homology-aware folds are not exchangeable, their spread is as large as either addition measured on them, and averaging it away costs roughly 200 GPU-hours per configuration.

2 Related work

Most predictors of proteolytic cleavage are specific by construction. One line covers a fixed set of proteases with known recognition motifs, one enzyme or family at a time: ProP [Duckert et al., 2004], PROSPER and its successors [Song et al., 2012, Li et al., 2023], DeepCleave [Li et al., 2020a],

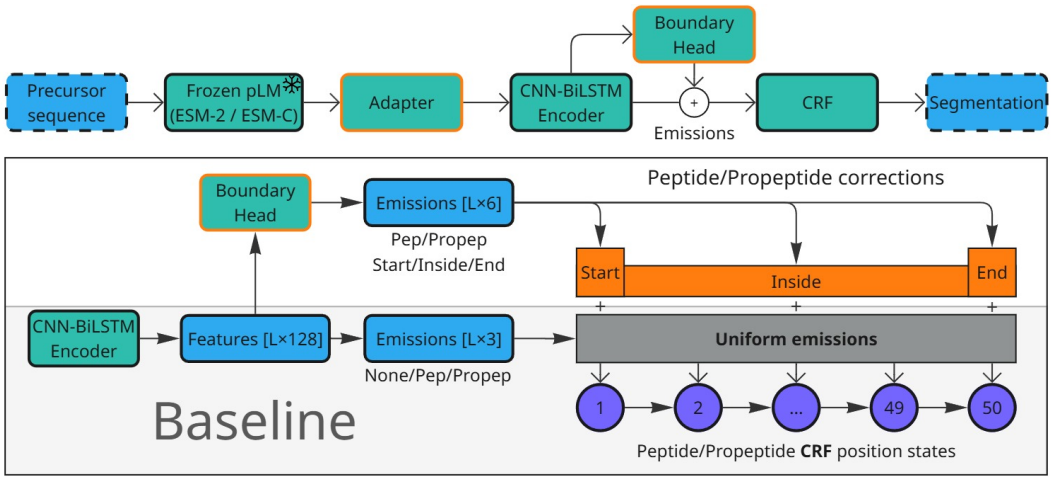

Figure 2: The pipeline from sequence to decoded segments, and where the two additions change it (top), with a zoom into the emission step (bottom). The adapter re-projects the frozen pLM embedding before the CNN–BiLSTM encoder. The boundary head reads the encoder output and adds a position-specific correction to the per-state CRF emissions before decoding. In the baseline (bottom, shaded) a single 3-way emission is repeated across every state of a given type; the boundary head produces a start/inside/end correction per type on top of it, so the first and last states of a segment no longer see the same evidence as its interior.

Procleave [Li et al., 2020b]. A second targets one product class in one kind of organism, most often neuropeptides, and scores cleavage only near basic residues [Southey et al., 2006, Wang et al., 2024]. Neither answers what an organism's full complement of proteases does to a given precursor.

General-purpose models instead train a compact head on frozen protein-language-model embeddings [Lin et al., 2023, ESM Team, 2024], but they mostly score peptides that have already been excised [Du et al., 2024, Zhu et al., 2025] rather than locating them; the pre-pLM PeptideLocator [Mooney et al., 2013] returns a per-residue similarity heatmap, not a segmentation. DeepPeptide [Teufel et al., 2023b] is, to our knowledge, the only model that segments a precursor into typed peptide and propeptide spans, and it also established the homology-partitioned benchmark used here [Teufel et al., 2023a], so we take both its architecture and its data pipeline as our starting point. Appendix A reviews this literature in more detail.

3 Method

The base architecture. DeepPeptide passes a frozen pLM embedding through a CNN–BiLSTM stack and decodes it with a linear-chain CRF whose state set expands each of $\{\mathsf{Peptide}, \mathsf{Propeptide}\}$ into a chain of up to 50 position states, 101 in all, so that a legal path can only realize a contiguous segment of admissible length and the model is trained to produce segments rather than independent residue labels. The encoder, however, computes one emission per label and shares it across all 50 states of that label: the decoder is boundary-aware, the features feeding it are not. The two additions close that gap from opposite ends of the pipeline (Figure 2).

Boundary head. A small feed-forward block (LayerNorm, Linear, GELU, Linear, hidden size 64) is applied to the contextual per-residue features. For each position and each segment type it emits three numbers, scoring how much that position looks like a segment start, an interior position, or a segment end, and these are added to the emissions of the corresponding position-fixed CRF states. The final linear layer is initialized to zero, so training begins at exactly the base model and the head only learns corrections that lower the loss. It costs 4,678 parameters on top of a trainable stack of 224,710, next to a frozen pLM of 650 million.

Adapter. The second addition leaves the decoder alone and acts on the input. Before the per-residue embedding reaches the CNN–BiLSTM it passes through LayerNorm, Linear, GELU, which re-projects it and reduces its width (1280 to 256 for ESM-2). A pLM embedding is trained on

masked-residue recovery, not on cleavage-site localization, and its feature distribution suits neither this task nor the small head that consumes it; a trainable re-projection in front of a frozen encoder is the standard remedy. It is the more expensive of the two additions: the projection itself holds 331,008 parameters, partly offset because the convolutions downstream now read 256 channels instead of 1280, for a net $+232{,}704$.

The two additions intervene at different points, so we test them separately and together, giving the 2×2 factorial in Table 2. Code, the eight run configurations and a synthetic smoke test are available at `https://anonymous.4open.science/r/cleavage-site-segmentation-5851`.

4 Data and evaluation

4.1 Dataset

A precursor $x = (a_1, \ldots, a_L)$ is labelled per residue with $y_t \in \{\mathsf{None}, \mathsf{Peptide}, \mathsf{Propeptide}\}$, and contiguous runs of a label form typed segments. The task is to recover those segments: which stretches of the precursor become mature peptides after cleavage, and which become propeptides that are excised and discarded.

DeepPeptide built its dataset from `PEPTIDE` and `PROPEP` annotations in the 2022 Swiss-Prot release. We rebuilt it with the same pipeline on the 2026 release [UniProt Consortium, 2025]. The collection grows from 8,449 proteins to 9,619, of which 1,178 were absent in 2022, and 8,897 of them carry ESM-2 embeddings and enter the five folds used here. Folds come from GraphPart [Teufel et al., 2023a] at a 30% pairwise-identity ceiling, balanced by cleavage-motif class. Segment-length filtering, motif balancing and the full composition of the rebuild are given in Appendix B.

4.2 Evaluation criterion

Peptides and propeptides are matched separately. A prediction is a true positive when its start and its end both fall within $\pm\tau$ residues of a true segment of the same type; unmatched predictions are false positives, unmatched true segments false negatives. We keep $\tau = 3$ as the headline setting, following the original evaluation: annotated cleavage sites carry a few residues of experimental uncertainty, so a prediction that close is practically an exact hit. Sweeping τ down to zero then asks a second question, not whether a peptide was found but how precisely its ends were placed.

The reference implementation of this criterion carries an upstream variable-shadowing bug that marks the wrong true segment as matched; everything reported here is re-scored with a corrected matcher (Appendix C).

Agreement with the published baseline. DeepPeptide reports precision 0.68 and recall 0.49 at ± 3 under nested cross-validation on the 2022 data, an implied F1 of 0.570. The same architecture on our 2026 rebuild reaches 0.573 ± 0.025 (Table 2). The agreement in F1 is closer than the comparison deserves, since the releases and the matchers differ and we trade precision for recall, but it does say the rebuild did not break the baseline.

4.3 Nested cross-validation and its cost

We evaluate every configuration with the 5×4 nested cross-validation DeepPeptide itself used. For each of five outer folds o, four models are trained on three of the remaining folds and validated on the fourth, then tested on o, which they never saw. An outer fold's score is the mean over its four inner models and the estimate is the mean over the five outer folds. The reported spread is the standard deviation across outer folds, not across all 20 cells, since four models sharing an outer fold also share a test set and are not independent repeats.

Fold exchangeability. Homology-aware partitioning is what makes the benchmark meaningful and also what makes the folds unequal. GraphPart keeps homologs together, and in this dataset entire taxa *are* homology clusters, venom families above all. Table 1 shows the result: folds differ in size by a factor of two, and a genus contributing several hundred proteins can be absent from one and dominant in another. Motif balancing, which the original pipeline does, does not touch this axis.

Table 1: Proteins per outer fold, and how three frequent genera distribute across them. *Cyriopagopus* and *Lycosa* are spider venoms, *Conus* a cone snail; each is a homology cluster that GraphPart is obliged to keep intact. *Homo*, by contrast, is spread evenly (71/81/66/130/84).

	total	**f0**	**f1**	**f2**	**f3**	**f4**
All proteins	8897	1558	2572	1263	2025	1479
Conus	714	160	313	16	149	76
Cyriopagopus	293	0	62	45	9	177
Lycosa	163	5	42	10	106	0

Table 2: 5×4 nested cross-validation with the corrected matcher, mean $\pm$ std. over the five outer folds. Precision and recall are at the headline ± 3 tolerance; the four F1 columns tighten the boundary-match tolerance to an exact residue. *Growth* is how much a row's F1 advantage over the ESM-2 base widens between ± 3 and an exact match, so a value near zero would mean its tolerance curve is a parallel translation of the baseline.

		at ± 3		**F1 by tolerance**				
Boundary	**Adapter**	**P**	**R**	± 3	± 2	± 1	exact	**growth**
ESM-2								
—	—	0.615 ± 0.020	0.538 ± 0.043	0.573 ± 0.025	0.540	0.461	0.345	—
✓	—	0.673 ± 0.029	0.538 ± 0.036	0.597 ± 0.026	0.566	0.488	0.384	+0.015
—	✓	0.628 ± 0.019	0.579 ± 0.040	0.602 ± 0.026	0.572	0.491	0.396	+0.022
✓	✓	0.667 ± 0.032	0.598 ± 0.026	0.630 ± 0.021	0.599	0.518	0.432	+0.029
ESM-C 6B								
—	—	0.620 ± 0.017	0.560 ± 0.027	0.588 ± 0.016	0.558	0.483	0.371	+0.010

This puts a floor under how small an effect a single split can resolve. The base architecture scores 0.538, 0.559, 0.574, 0.594 and 0.599 on the five outer folds: a range of 0.061, which is two and a half times the $+0.024$ effect of the boundary head that the same experiment resolves once averaged over folds. Under an earlier single-split protocol we ran during development, that same modification measured as no effect at all, with a confidence interval covering zero (Appendix D). The modification did not change; the resolution did.

The case against a third level of nesting. The protocol still selects architectures on outer-fold scores, which a sealed design would forbid; closing that needs a third held-out level. One cell costs about 10 GPU-hours, so a configuration of 20 cells costs roughly 200 and the full $2 \times 2 \times 2$ grid roughly 1,600, which is weeks of continuous single-GPU compute for the configurations reported here. A third level multiplies that again while cutting the training fraction from 3/5 to 2/5 of an already small dataset. At this size the protocol cannot be made both clean and affordable, so we state the trade-off rather than leave it implicit.

5 Results

Table 2 reports the 2×2 factorial on ESM-2, with the base architecture on ESM-C 6B as a capacity control, all under the protocol of Section 4.3 with the corrected matcher.

The two additions act on different errors. Each improves F1 by a similar amount on its own, and together they give $+0.058$, close to the sum of the parts. Splitting F1 into its components explains why. The boundary head is almost purely a precision effect: precision rises by 0.057 while recall does not move at all ($+0.0002$). The adapter is mostly a recall effect: recall rises by 0.041 against 0.013 of precision. Applied together they recover both, $+0.052$ precision and $+0.061$ recall. They add up because they act on different error types. One suppresses spurious segments; the other finds segments the base model misses.

Tightening the tolerance. Figure 1 and the F1 columns of Table 2 sweep τ from ± 3 to an exact match. Absolute F1 falls steeply for every model: the best configuration goes from 0.630 at ± 3 to 0.432 at the exact residue. Placing a cleavage site exactly is still an open problem.

A model that is better overall retains a different *fraction* of its ± 3 score even when no boundary is placed better, so we compare absolute gaps rather than ratios. Every gap widens as the tolerance tightens, from $+0.024$ to $+0.039$ for the boundary head, $+0.029$ to $+0.051$ for the adapter and $+0.058$ to $+0.086$ for the two together, and each is nearly flat from ± 3 to ± 1 before opening up at the last step: what separates these models is specifically whether they hit the exact residue.

A widening gap is still not proof of better localization, since a model that finds more segments also gains ground at every tolerance. To hold detection fixed we compared each variant against the base on the true segments that *both* localize within ± 3, matched cell by cell and segment by segment. On that set the adapter is the better sharpener: the share placed exactly right rises from 0.608 to 0.667, positive on all five outer folds ($+0.057 \pm 0.027$). The boundary head moves it from 0.622 to 0.653, but only four folds agree and the spread (± 0.040) exceeds the effect, so we report the direction and not the magnitude. Together they reach 0.697 from 0.611, positive on every fold ($+0.081 \pm 0.038$); segment counts are in Appendix D.4. Read with the precision/recall split above, this gives each block a job: on ESM-2 the head filters spurious segments, while the adapter both finds more segments and places their boundaries more precisely.

A richer embedding is not enough on its own. The base architecture on ESM-C 6B reaches 0.588 ± 0.016 against 0.573 ± 0.025 on ESM-2. The intervals overlap, so an embedding twice as wide buys no confirmed improvement on its own, while either architectural addition on the narrower one does. It does show the same tolerance signature as the additions, $+0.015$ at ± 3 against $+0.025$ at an exact match, so what capacity buys here is boundary precision rather than more segments found. Whether the additions gain more on the richer embedding is open. Under the earlier single-split protocol the boundary head on ESM-C 6B was worth $+0.053$ and $+0.067$ F1 on the two held-out folds, and pooled over both it raised recall ($+0.045$) as well as precision ($+0.081$), unlike on ESM-2, where it buys precision without gaining recall (Appendix D). That is the protocol whose resolution we have just argued against, so we record this as a hypothesis, not a result.

6 Conclusion

Two small additions to the DeepPeptide recipe, a zero-initialized boundary head at the decoder and a lightweight adapter at the input, improve segment F1 by $+0.058$ together and by $+0.024$ and $+0.029$ on their own. They combine because they act on different errors, the head suppressing spurious segments and the adapter finding more real ones, and both place segment boundaries more precisely, so their advantage widens as the match tolerance tightens.

The evaluation carries a second message: homology-aware partitioning leaves the folds taxonomically non-exchangeable, their spread is as large as either addition measured on them, and averaging it away, at about 200 GPU-hours per configuration, is what makes those effects visible at all.

Limitations. The reported spread covers fold composition only, not seed variation, so the true uncertainty is wider. The additions were evaluated under nested cross-validation on ESM-2 only; on ESM-C 6B they have single-split evidence (Appendix D). The protocol still selects architectures using outer-fold scores rather than a sealed third level, for the reasons given in Section 4.3. A larger set of ideas from earlier development, including a structural channel, an auxiliary bond loss, a telescopic segment CRF and LoRA fine-tuning, was only ever tested under the weaker protocol and is reported in Appendix D as unconfirmed.

References

José Juan Almagro Armenteros, Marco Salvatore, Olof Emanuelsson, Ole Winther, Gunnar von Heijne, Arne Elofsson, and Henrik Nielsen. Detecting sequence signals in targeting peptides using deep learning. *Life Science Alliance*, 2(5):e201900429, 2019. doi: 10.26508/lsa.201900429.

Zhenjiao Du, Xingjian Ding, William Hsu, Arslan Munir, Yixiang Xu, and Yonghui Li. pLM4ACE: a protein language model based predictor for antihypertensive peptide screening. *Food Chemistry*, 431:137162, 2024. doi: 10.1016/j.foodchem.2023.137162.

Peter Duckert, Søren Brunak, and Nikolaj Blom. Prediction of proprotein convertase cleavage sites. *Protein Engineering, Design and Selection*, 17(1):107–112, 2004. doi: 10.1093/protein/gzh013.

Ahmed Elnaggar, Hazem Essam, Wafaa Salah-Eldin, Walid Moustafa, Mohamed Elkerdawy, Charlotte Rochereau, and Burkhard Rost. Ankh: Optimized protein language model unlocks general-purpose modelling, 2023. URL https://arxiv.org/abs/2301.06568.

ESM Team. Esm cambrian: Revealing the mysteries of proteins with unsupervised learning. https://www.evolutionaryscale.ai/blog/esm-cambrian, December 2024.

Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *bioRxiv*, 2024. doi: 10.1101/2024.07.01.600583. URL https://www.biorxiv.org/content/early/2024/12/31/2024.07.01.600583.

Michael Heinzinger, Konstantin Weissenow, Joaquin Gomez Sanchez, Adrian Henkel, Milot Mirdita, Martin Steinegger, and Burkhard Rost. ProstT5: bilingual language model for protein sequence and structure. *NAR Genomics and Bioinformatics*, 6(4):lqae150, 2024. doi: 10.1093/nargab/lqae150.

Fuyi Li, Jinxiang Chen, André Leier, Tatiana Marquez-Lago, Quanzhong Liu, Yanze Wang, Jerico Revote, A. Ian Smith, Tatsuya Akutsu, Geoffrey I. Webb, Lukasz Kurgan, and Jiangning Song. DeepCleave: a deep learning predictor for caspase and matrix metalloprotease substrates and cleavage sites. *Bioinformatics*, 36(4):1057–1065, 2020a. doi: 10.1093/bioinformatics/btz721.

Fuyi Li, André Leier, Quanzhong Liu, Yanan Wang, Dongxu Xiang, Tatsuya Akutsu, et al. Procleave: predicting protease-specific substrate cleavage sites by combining sequence and structural information. *Genomics, Proteomics & Bioinformatics*, 18(1):52–64, 2020b. doi: 10.1016/j.gpb.2019.08.002.

Fuyi Li, Cong Wang, Xudong Guo, Tatsuya Akutsu, Geoffrey I. Webb, Lachlan J. M. Coin, Lukasz Kurgan, and Jiangning Song. ProsperousPlus: a one-stop and comprehensive platform for accurate protease-specific substrate cleavage prediction and machine-learning model construction. *Briefings in Bioinformatics*, 24(6):bbad372, 2023. doi: 10.1093/bib/bbad372.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

Catherine Mooney, Niall J. Haslam, Gianluca Pollastri, and Denis C. Shields. Towards the improved discovery and design of functional peptides: common features of diverse classes permit generalized prediction of bioactivity. *PLoS ONE*, 7(10):e45012, 2012. doi: 10.1371/journal.pone.0045012.

Catherine Mooney, Niall J. Haslam, Thérèse A. Holton, Gianluca Pollastri, and Denis C. Shields. PeptideLocator: prediction of bioactive peptides in protein sequences. *Bioinformatics*, 29(9):1120–1126, 2013. doi: 10.1093/bioinformatics/btt103.

Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021.

Jiangning Song, Hao Tan, Andrew J. Perry, Tatsuya Akutsu, Geoffrey I. Webb, James C. Whisstock, and Robert N. Pike. PROSPER: an integrated feature-based tool for predicting protease substrate cleavage sites. *PLoS ONE*, 7(11):e50300, 2012. doi: 10.1371/journal.pone.0050300.

Jiangning Song et al. PROSPERous: high-throughput prediction of substrate cleavage sites for 90 proteases with improved accuracy. *Bioinformatics*, 34(4):684–687, 2018. doi: 10.1093/bioinformatics/btx670.

Bruce R. Southey, Andinet Amare, Tyler A. Zimmerman, Sandra L. Rodriguez-Zas, and Jonathan V. Sweedler. NeuroPred: a tool to predict cleavage sites in neuropeptide precursors and provide the masses of the resulting peptides. *Nucleic Acids Research*, 34(Web Server issue):W267–W272, 2006. doi: 10.1093/nar/gkl161.

Felix Teufel, José Juan Almagro Armenteros, Alexander Rosenberg Johansen, Magnús Halldór Gíslason, Silas Irby Pihl, Konstantinos D. Tsirigos, Ole Winther, Søren Brunak, Gunnar von Heijne, and Henrik Nielsen. SignalP 6.0 predicts all five types of signal peptides using protein language models. *Nature Biotechnology*, 40(7):1023–1025, 2022. doi: 10.1038/s41587-021-01156-3.

Felix Teufel, Magnús Halldór Gíslason, José Juan Almagro Armenteros, Alexander Rosenberg Johansen, Ole Winther, and Henrik Nielsen. GraphPart: homology partitioning for biological sequence analysis. *NAR Genomics and Bioinformatics*, 5(4):lqad088, 2023a. doi: 10.1093/nargab/lqad088.

Felix Teufel, Jan Christian Refsgaard, Christian Toft Madsen, Carsten Stahlhut, Mads Grønborg, Ole Winther, and Dennis Madsen. Deeppeptide predicts cleaved peptides in proteins using conditional random fields. *Bioinformatics*, 39(10):btad616, 10 2023b. ISSN 1367-4811. doi: 10.1093/bioinformatics/btad616. URL `https://doi.org/10.1093/bioinformatics/btad616`.

UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025. doi: 10.1093/nar/gkae1010. URL `https://academic.oup.com/nar/article/53/D1/D609/7902999`.

Michel van Kempen, Stephanie S. Kim, Charlotte Tumescheit, Milot Mirdita, Jeongjae Lee, Cameron L. M. Gilchrist, Johannes Söding, and Martin Steinegger. Fast and accurate protein structure search with Foldseek. *Nature Biotechnology*, 42:243–246, 2024. doi: 10.1038/s41587-023-01773-0.

Lei Wang, Chen Huang, Mingxia Wang, Zhidong Xue, and Yan Wang. NeuroPred-PLM: an interpretable and robust model for neuropeptide prediction by protein language model. *Briefings in Bioinformatics*, 24(2):bbad077, 2023. doi: 10.1093/bib/bbad077.

Lei Wang, Zilu Zeng, Zhidong Xue, and Yan Wang. DeepNeuropePred: a robust and universal tool to predict cleavage sites from neuropeptide precursors by protein language model. *Computational and Structural Biotechnology Journal*, 23:309–315, 2024. doi: 10.1016/j.csbj.2023.12.004.

M. Zhang, J. Zhou, X. Wang, et al. DeepBP: ensemble deep learning strategy for bioactive peptide prediction. *BMC Bioinformatics*, 25:352, 2024. doi: 10.1186/s12859-024-05974-5.

Lei Zhu, Hao Sun, and Sen Yang. BPFun: a deep learning framework for bioactive peptide function prediction using multi-label strategy by transformer-driven and sequence rich intrinsic information. *BMC Bioinformatics*, 26(1):187, 2025. doi: 10.1186/s12859-025-06190-5.

Impact Statement

This paper improves a computational tool for predicting proteolytic cleavage products from protein sequence. Potential applications include interpreting proteomics data, informing vaccine and peptide-drug design, and studying disease-associated proteolysis; we are not aware of societal risks specific to this work beyond those generally associated with more accurate protein sequence analysis tools.

Appendix

A Extended review of prior work

Section 2 surveys this literature in the space a workshop paper allows. This appendix gives the same material in the detail a reader unfamiliar with the task will want.

Methods for predicting proteolytic peptides fall into three broad classes.

Protease-specific models. These predict cleavage only for a limited set of enzymes, most of which have a known recognition motif. ProP [Duckert et al., 2004] targets cleavage by the PACE/PC family in animals and plants; PROSPER [Song et al., 2012] and its successor PROSPERous [Song et al., 2018] predict cleavage for one chosen enzyme at a time, as does the later ProsperousPlus [Li et al., 2023]; DeepCleave [Li et al., 2020a] is restricted to caspases and matrix metalloproteinases; and Procleave [Li et al., 2020b] likewise handles one enzyme at a time and additionally requires 3D structural input. The shared limitation of this class is scope: in practice, one is usually interested in the combined effect of all proteases active in an organism, not any single enzyme.

Organism- or peptide-type-specific models. A separate class targets neuropeptides or other bioactive fragments specific to a given organism. NeuroPred [Southey et al., 2006] and the species-agnostic DeepNeuropePred [Wang et al., 2024] only detect cleavage near a handful of basic residues; NeuroPred-PLM [Wang et al., 2023] does not localize cleavage sites at all but classifies an already-excised sequence. A related group of models finds signal subsequences rather than any bioactive peptide: SignalP [Teufel et al., 2022] and TargetP [Almagro Armenteros et al., 2019] identify the type of signal peptide, which indicates where a protein is trafficked.

General-purpose models on pLM embeddings. Protein language models (pLMs) are transformers trained without supervision on large sequence corpora, typically by masked-residue recovery, and they produce transferable per-residue embeddings: ESM-2 [Lin et al., 2023], the earlier ESM line [Rives et al., 2021], ESM Cambrian [ESM Team, 2024, Hayes et al., 2024], and Ankh [Elnaggar et al., 2023]. Compact task-specific heads are then trained on top of these frozen embeddings. Besides **DeepPeptide** [Teufel et al., 2023b] itself, this recipe underlies DeepNeuropePred, NeuroPred-PLM, and SignalP 6, as well as models that score already-excised peptides rather than finding them: pLM4ACE [Du et al., 2024] predicts ACE-inhibitory activity, and BPFun [Zhu et al., 2025] and DeepBP [Zhang et al., 2024] predict broader bioactivity, building on earlier pre-pLM work such as PeptideRanker [Mooney et al., 2012]. Before pLMs, PeptideLocator [Mooney et al., 2013] addressed a problem close to ours, but its output is a per-residue heatmap of similarity to known bioactive peptides rather than an explicit segmentation, and it was the main point of comparison in the original DeepPeptide paper.

Baseline architecture: DeepPeptide. Since we build directly on it throughout this paper (Section 3), DeepPeptide's architecture is worth describing in detail rather than treating as a black box. It first encodes a precursor sequence with a frozen pLM (ESM-2), producing a per-residue embedding that is not fine-tuned during training. This embedding is refined by a convolution, a single-layer bidirectional LSTM and a second convolution. The published description does not fix the widths: they are searched with Optuna inside the inner cross-validation loop. The configuration we train throughout uses 32 convolutional filters of width 3 and an LSTM hidden size of 64 per direction, giving 224,710 trainable parameters for the base model. These features are consumed by a linear-chain CRF decoder whose state set is extended well beyond the three raw labels $\{\mathsf{None}, \mathsf{Peptide}, \mathsf{Propeptide}\}$: each of the two positive labels is expanded into a chain of states long enough to represent segments up to a fixed maximum length (101 states in total for the 50-residue cap used here, Section 4.1), so that a legal path through the CRF can only realize a contiguous segment of valid length, with dedicated states marking its first and last positions. This makes segmentation, rather than independent per-residue classification, the object the model is directly optimized for – and it is also where the gap described in Section 1 lives: every state in this extended set that shares a type receives the same emission from the encoder, regardless of whether it marks the start, interior, or end of a segment.

Baseline dataset for this task. Beyond its architecture, DeepPeptide also established the data resource this line of work relies on: precursor sequences from UniProtKB/Swiss-Prot annotated with `PEPTIDE` and `PROPEP` feature types, partitioned into homology-aware folds with GraphPart [Teufel et al., 2023a] at a 30% pairwise-identity threshold and additionally balanced by cleavage-flanking motif, then evaluated with the same family of nested cross-validation we adopt (Section 4.3). This

gave the field a standardized, already homology-controlled benchmark rather than an ad hoc collection of previously published peptide lists, and both the curated data and the construction pipeline that produced it are public. Section 4.1 describes what we do with that resource: rebuilding it on a current UniProtKB/Swiss-Prot release so that it has enough scale and coverage for 5×4 nested cross-validation.

Among these, DeepPeptide is, to our knowledge, the only model that poses the task as full segmentation of the precursor into typed peptide and propeptide segments, and it is the strongest available baseline for this formulation. We therefore use its architecture and its dataset-construction methodology as the starting point for this work, rather than treating it as a system to audit: our contribution is an architectural addition to this general recipe (Section 3), evaluated under a more conservative protocol (Section 4) on data rebuilt at greater scale (Section 4.1).

B Task formulation and dataset composition

Table 3: Dataset composition: the 2022 release used by the original DeepPeptide against the 2026 release rebuilt here. Counts are before homology partitioning.

	2022	**2026**
Proteins	8449	9619
of which absent in 2022	—	1178 (≈12%)
Peptide segments	6372	7431
Propeptide segments	8211	9140
Median peptide length (residues)	20–21	20–21

Two filters from the original pipeline are kept unchanged. Segments shorter than 5 or longer than 50 residues are dropped: the CRF's state count fixes the upper bound, and a segment shorter than five residues is barely scoreable at a ± 3 tolerance. Folds are then balanced by cleavage-motif class, obtained by k-means ($k = 50$) on ESM-2 embeddings of the four residues flanking each annotated boundary, on top of the 30% pairwise-identity ceiling GraphPart enforces between folds.

B.1 Problem formulation

A precursor protein is a sequence of amino acids

$$x = (a_1, a_2, \ldots, a_L), \qquad a_t \in \Omega,$$

where Ω is the 20-letter amino acid alphabet and L is the sequence length. The task is to predict which contiguous stretches of x become mature **peptides** after cleavage, and which become **propeptides** – auxiliary stretches that are excised during maturation and removed, without themselves being a final active product. Formally, each position is assigned a label

$$y_t \in \{\mathsf{None},\ \mathsf{Peptide},\ \mathsf{Propeptide}\},$$

and contiguous runs of the same label form typed **segments** (start, end) pairs:

$$\underbrace{\texttt{MGK}}_{\text{None}}\ \underbrace{\texttt{RALR}}_{\text{Propeptide}}\ \underbrace{\texttt{RSGK}}_{\text{Peptide}}\ \underbrace{\texttt{VR}}_{\text{None}}\ \underbrace{\texttt{NL}}_{\text{Peptide}}$$

How a predicted segment is scored against the ground truth – including the tolerance used for boundary matching – is defined later, in Section 4.2, alongside the rest of the evaluation protocol; it is an evaluation-time decision rather than part of the task definition itself.

C Metric implementation bug

While auditing the reference implementation of the matching criterion (Section 4.2), we found a bug in the function that implements it (`get_counts_for_protein`, in the DeepPeptide codebase), reproduced below in simplified form:

```
for idx, row in true_df.iterrows():      # idx: true-segment index
    true_start, true_stop = row['start'], row['stop']
```

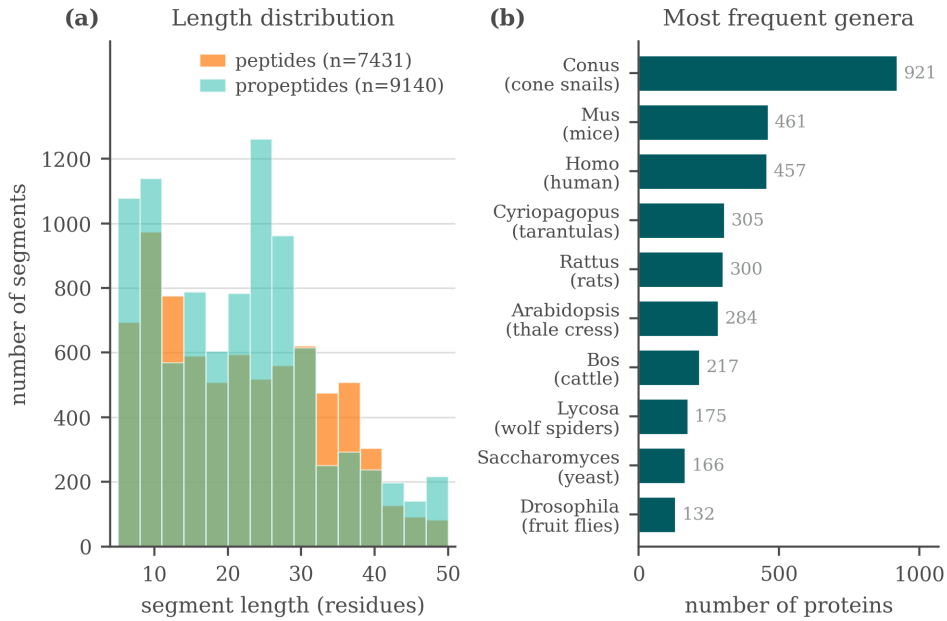

Figure 3: Composition of the rebuilt 2026 dataset: segment length, type, and genus distributions.

```
    for idx, row in pred_df.iterrows():  # idx overwritten with pred index
        pred_start, pred_stop = row['start'], row['stop']
        ...
        if start_match and stop_match:
            true_df.loc[idx, 'matched'] = True  # BUG: idx is now the
            pred_df.loc[idx, 'matched'] = True   # predicted index, not the true one
            break
```

The outer loop iterates over true segments and the inner loop over predicted segments, but both reuse the loop variable `idx`. By the time a match is found, `idx` refers to the predicted segment, so `true_df.loc[idx, 'matched']` marks the wrong true segment. The effect is not large on average (it is partly self-correcting), but it costs a true positive whenever it fires, so both recall and precision come out low, and F1 with them. Across the nested-CV grid the correction raises recall by 0.023 to 0.029 and precision by 0.007 to 0.010. The loss concentrates in proteins where the model predicts more segments than the protein actually has. This bug is present in the upstream DeepPeptide code, not something we introduced.

We did not silently patch this function in place, since that would shift absolute numbers without documenting it. Instead we implemented a separate, corrected matcher and used it to re-score the results in this paper: the earlier single-split runs reported in Appendix D, except two whose model classes no longer exist in the code and whose checkpoints cannot be rebuilt, and the full 5×4 nested-cross-validation grid in Table 2 (all 100 completed cells across the five configurations), by re-running test-partition inference from each cell's saved checkpoint on the machine that held them. Every table in the main text uses the corrected matcher; the original (buggy) numbers are kept alongside in the released run artifacts for direct comparison with the published methodology, not reproduced here.

Re-scoring the grid also recomputed the original, buggy matcher on the same decoded segments, so that each cell could be checked against the number already published for it. The two agree to 0.0016 F1 at worst and to 3×10^{-7} at the median, and the aggregate reproduces the published summary exactly.

Across the five configurations the correction shifts mean F1 by $+0.017$ to $+0.020$, always upward, since the bug was discarding true positives. On ESM-2 it leaves the ordering of the four configurations intact: base $<$ boundary head $<$ adapter $<$ combined, before and after correction. That ordering is not robust, and we would not build an argument on it. The boundary-head/adapter gap is 0.005 against a fold-level standard deviation of 0.026; per outer fold the full ordering holds in four folds of five after correction and three of five before it, and on peptides alone it holds in one

fold of five. Only the base configuration exists on ESM-C 6B, so no ordering can be stated for that embedding.

D Earlier single-split protocol: unconfirmed observations

This appendix reports findings from an earlier stage of the project, obtained under a single-split protocol that predates the 5×4 nested cross-validation adopted for the confirmed results in Section 5. We report them here, explicitly separated from the main results, both because they motivated the switch to nested CV and because several of them describe architectural or data ideas that were never re-tested under the more rigorous protocol. None of the numbers below should be read with the same confidence as Table 2.

D.1 Protocol

Data was split into seven GraphPart folds (30% identity threshold, motif-balanced as in Section 4.1), with folds assigned fixed roles: four folds for training, one for validation (epoch selection), one for model selection (comparing architectures), and one originally intended as a fully sealed test fold. This separated architecture selection from evaluation, but the evaluation itself was still a single draw: one random assignment of roles, one pair of held-out folds. Several modifications changed the *sign* of their measured effect between the two held-out folds. Replacing ESM-2 with ESM-C 6B measured $+0.074$ on one and -0.011 on the other; ESM-C 6B against ESM-C 600M gave $+0.028$ and -0.021; the net effect of the 3Di channel gave $+0.015$ and -0.018. Effects that kept their sign still moved by a lot: the isolated gated adapter measured -0.001 on one fold and $+0.045$ on the other. Figure 4 shows one reason, namely that the folds differ substantially in segment-length composition. The two held-out folds turn out to be complementary in segment-length composition, fold 2 concentrated at 10–24 residues and fold 5 bimodal, so results below pool them (model-select $\cup$ sealed, $\approx$2,300 proteins) to reduce (but not eliminate) this instability; this pooling is why the "sealed" fold is not, in practice, a held-out test set independent of model selection.

D.2 Summary of tested modifications

Table 4 summarizes the modifications tested under this protocol, with the corrected matcher (Appendix C) applied throughout. Three of these rows were later re-tested under nested cross-validation: the boundary head, the adapter (Section 3), and the ESM-C 6B embedding swap. Two survived. The embedding swap did not: nested cross-validation puts ESM-C 6B at 0.588 ± 0.016 against ESM-2's 0.573 ± 0.025 (Table 2), overlapping intervals, where this table records a confident $+0.03$. Everything else below remains at the confidence of a single pooled split.

Table 4: Modifications tested under the earlier single-split protocol, with their measured effect on F1 (pooled held-out folds) and an informal verdict. None of these effect sizes should be compared directly to the nested-CV numbers in Table 2.

Modification	Type	Measured effect on F1	Verdict
ESM-C 6B instead of ESM-2	embedding swap	$+0.03$, unstable across folds	helps
Boundary head on ESM-C 6B	decoder addition	$+0.05$, $+0.07$, consistent	helps
Boundary head on ESM-2	decoder addition	≈ 0 (CI includes zero)	no effect
Structural channel (3Di)	extra input	propep. $+0.02$ / pep. -0.04, net trade-off	no effect
ESM-C 6B compression 2560→256	optimization	≈ 0, $10\times$ narrower input and $16\times$ fewer trainable parameters	no effect
Gated adapter on ESM-C 6B (pLM re-projection)	input adapter	$+0.022$, CI $[+0.008, +0.038]$	helps
Bond-prediction auxiliary loss (on ESM-C 6B)	extra loss	pep. -0.05, net -0.015; neutral on ESM-2	harmful
Telescopic segment CRF	decoder addition	≈ 0 on ESM-2, $+0.022$ on ESM-C 6B, no CI available	unresolved

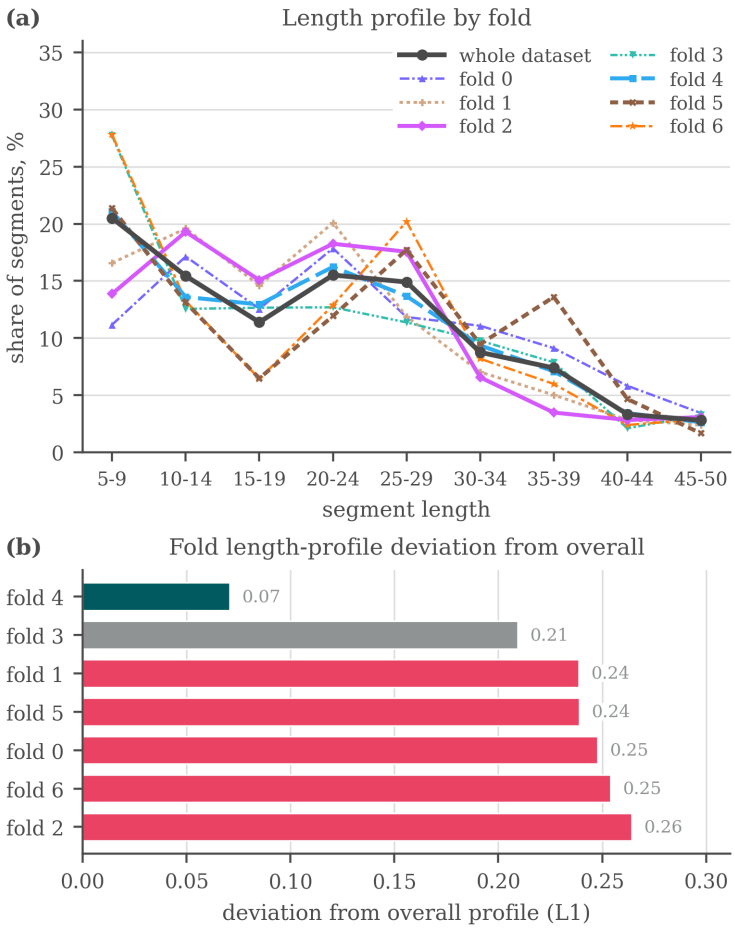

Figure 4: Why the folds of the earlier seven-fold split are not interchangeable: (a) the segment-length profile of each fold against the profile of the whole dataset, (b) the L_1 distance between the two. Fold 4 tracks the overall distribution closely (0.07) while five of the seven folds sit above 0.23. Balancing was done on cleavage motif and homology, not on segment length, so this axis was left free. The same holds for the five-fold split used in the main text, where the imbalance is taxonomic (Table 1).

Note the apparent tension with Table 2: under this earlier protocol, the boundary head on ESM-2 looked like it had no effect, while nested cross-validation resolved a confirmed $+0.024$ F1 gain for the same modification on the same embedding. We read this as evidence that the single-split protocol lacked the statistical power to resolve an effect of this size, not as a contradiction – and as the clearest illustration of why we do not treat single-split numbers as findings in this paper. Whether the boundary head's effect is specifically larger on richer embeddings (as the ESM-C 6B rows above suggest) is accordingly left as an open question, not stated as a conclusion here.

D.3 Additional exploratory modifications

A larger set of ideas was tried under the same single-split protocol and did not show a clear, consistent benefit; we list them for completeness and as candidates for future re-testing, without effect-size claims:

- **Known-peptide dictionary.** An Aho–Corasick index of peptides from the training data was used to flag substring matches in a query sequence, injected as a bonus on the CRF state emissions, as a bias on the start, inside and end emissions specifically, fused into the encoder hidden state, and concatenated with the embedding before the encoder. No

consistent benefit was observed, and the checkpoint for one of the variants can no longer be loaded.

- **Structural features.** Features extracted from predicted 3D structures (AlphaFold2 representations via the AFToolkit codebase, with several confidence-based filtering variants) and from the ProstT5 [Heinzinger et al., 2024] structural alphabet (3Di, as used in Foldseek [van Kempen et al., 2024]) were tested as additional input channels; results were mixed (Table 4).
- **LoRA fine-tuning.** Low-rank adaptation of the last few pLM layers, instead of fully frozen embeddings, scored 0.558 and 0.567 against 0.621 for the frozen baseline, all three on the 2022 release under its own five-fold split rather than the protocol of Appendix D. It also ran for fewer epochs in the same time budget, though we did not measure the cost directly.
- **Projector variants.** Multi-scale and multi-branch projectors between the embedding and the CNN–BiLSTM were tried as alternative re-projection schemes to the adapter of Section 3.
- **Structural-projection width and telescopic CRF.** The width of the structural-feature projection (16/32/48 units) left pooled F1 flat at 0.691, 0.697 and 0.693, the telescopic segment CRF, an alternative decoder formulation that scores whole segments through a relative-position head, came out level with the base decoder on ESM-2 and $+0.022$ ahead on ESM-C 6B. Per-protein outputs were not retained for either run, so neither number carries a confidence interval and we treat the modification as untested rather than as having no effect.

None of these modifications showed a consistent gain large enough, under the single-split protocol, to justify testing under nested cross-validation ahead of the two reported in the main text. The complete experiment log (including runs not summarized above) is maintained in the project repository rather than reproduced here, since it was collected under a protocol we no longer treat as sufficient evidence on its own.

D.4 Complete tolerance sweep

Table 5 repeats the tolerance sweep of Table 2 with the fold-level standard deviation on every cell. The paired localization comparison of Section 5 runs on the true segments that a variant and the base both place within ± 3: 26,730 segments for the boundary head, 29,051 for the adapter and 29,132 for the two together, pooled over the twenty cells of each configuration.

Table 5: Segment F1 by boundary-match tolerance, 5×4 nested cross-validation, corrected matcher, mean over the five outer folds.

Configuration	± 3	± 2	± 1	exact	**gap growth**
ESM-2, base	0.573	0.540	0.461	0.345	$+0.000$
ESM-2 + boundary head	0.597	0.566	0.488	0.384	$+0.015$
ESM-2 + adapter	0.602	0.572	0.491	0.396	$+0.022$
ESM-2 + boundary head + adapter	0.630	0.599	0.518	0.432	$+0.029$
ESM-C 6B, base	0.588	0.558	0.483	0.371	$+0.010$

D.5 Supplementary figures

The figures below are from the same earlier development stage as Table 4 and are included for completeness; none of them should be read as confirmed evidence in the sense of Section 5.

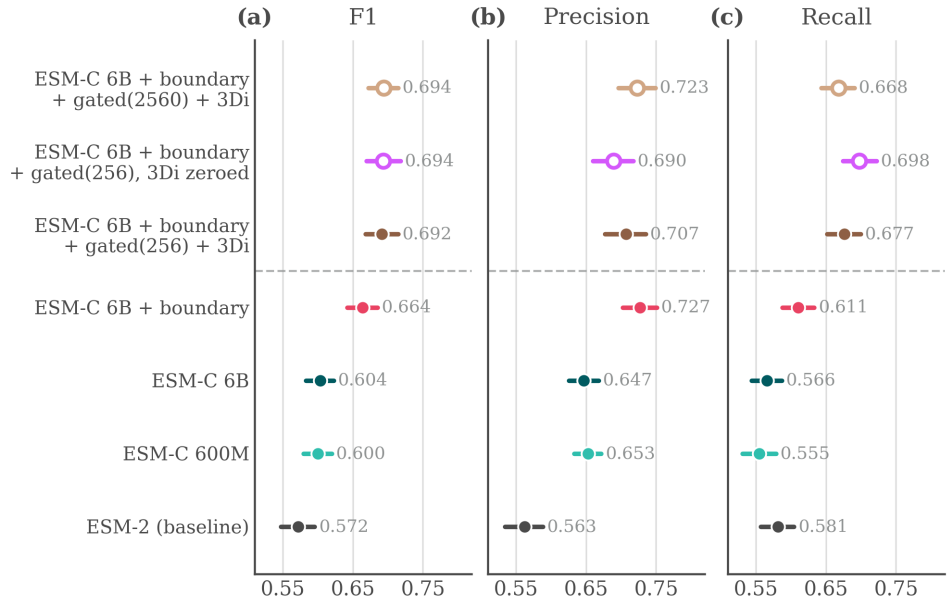

Figure 5: Single-split comparison on the pooled held-out folds at ±3: F1, precision and recall with bootstrap confidence intervals over 2,322 proteins. The top row is a gated-projector control at full width, not the configuration carried into the main text.

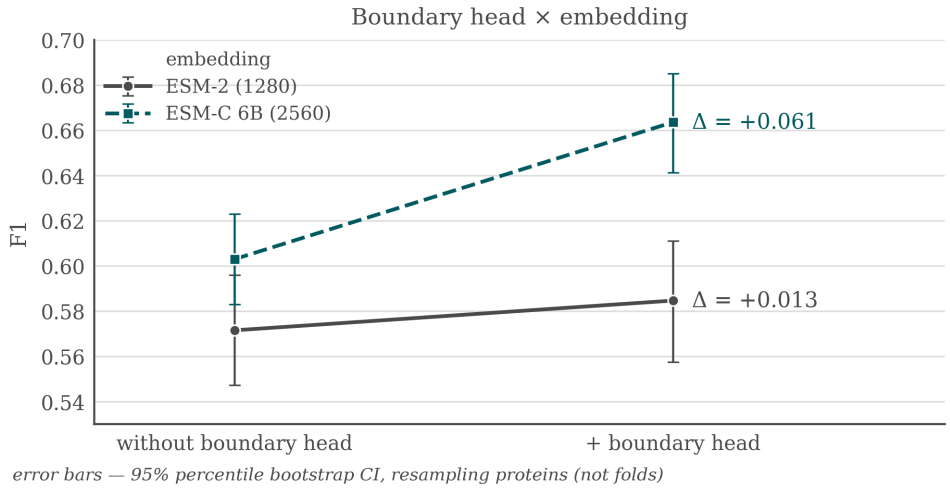

Figure 6: Boundary head × embedding interaction under the single-split protocol: F1 gain from adding the boundary head on top of ESM-2 vs. on top of ESM-C 6B. This is the earlier, unconfirmed version of the interaction question raised in Section 5.

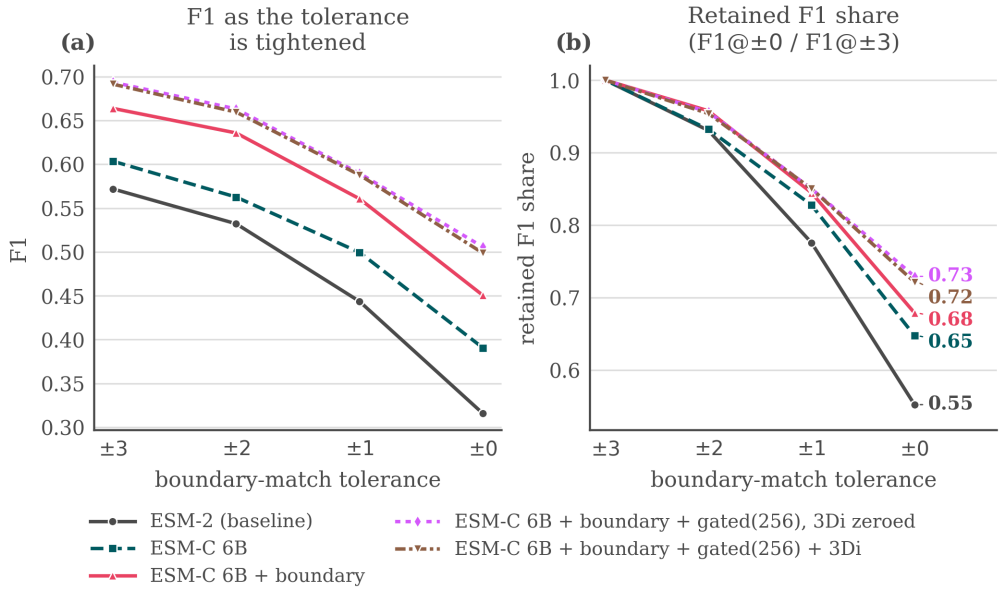

Figure 7: Quality as the matching tolerance is tightened from ±3 to an exact match (±0), single-split protocol, pooled held-out folds.

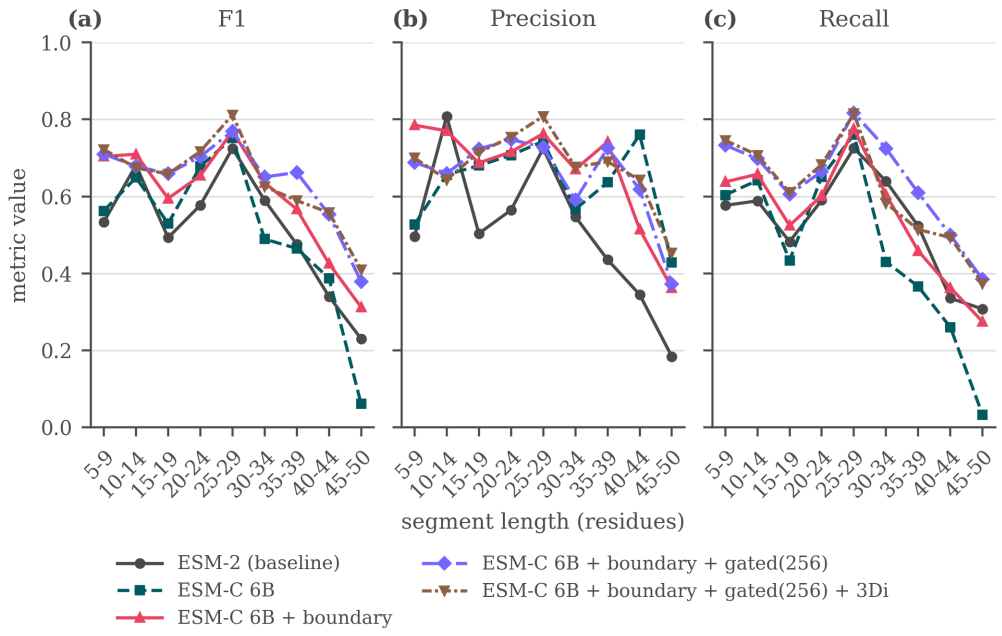

Figure 8: F1, precision, and recall as a function of segment length, single-split protocol, pooled held-out folds.

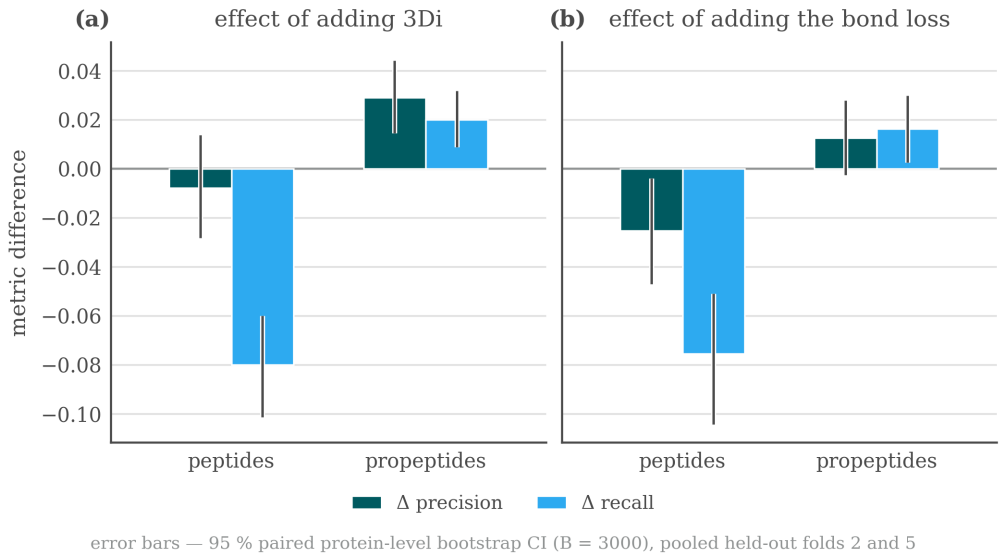

Figure 9: Precision/recall trade-offs by segment type when adding the 3Di channel or the bond-prediction loss, single-split protocol, pooled held-out folds.

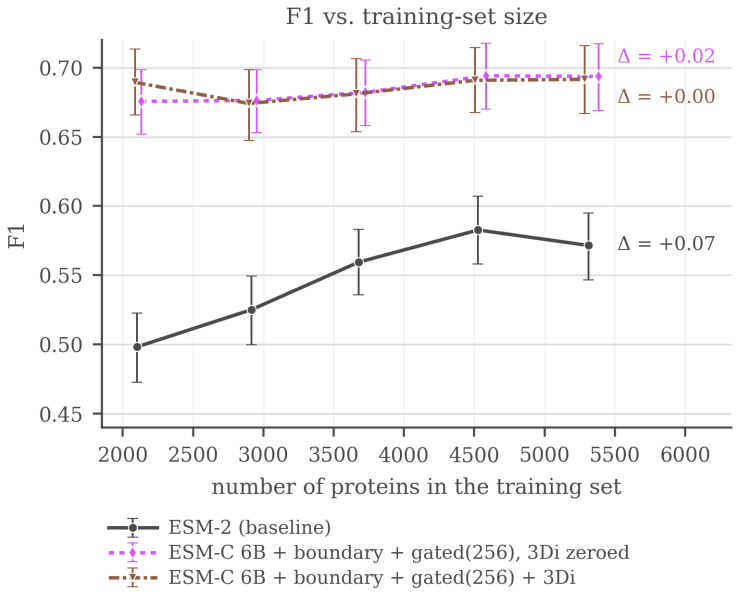

Figure 10: F1 (±3) against the number of training proteins, single-split protocol. ESM-2 rises from 0.498 at 40% of the training folds to 0.583 at 85%, then falls back to 0.572 at 100%, so it is still gaining over most of the range but not at the last point. The ESM-C 6B curves are flat from the smallest size tested, with movement smaller than the bootstrap interval, so nothing is observed saturating: they simply never climb.

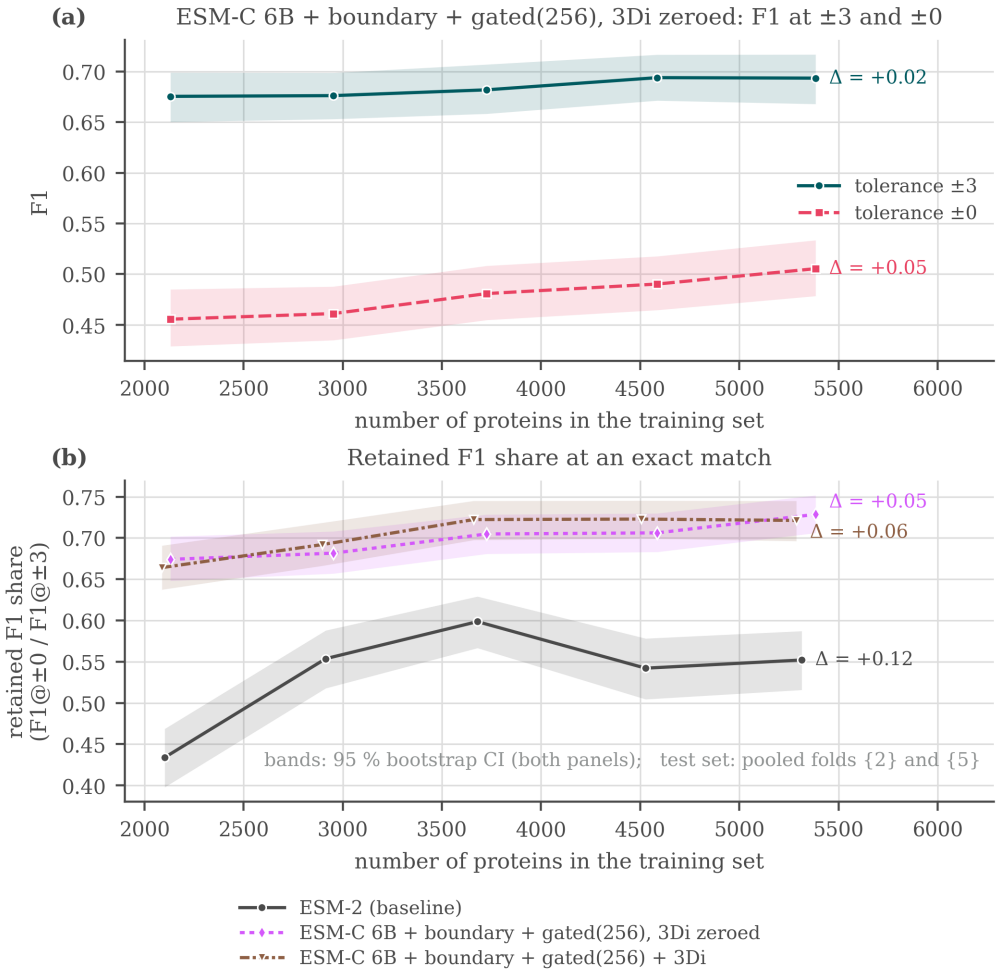

Figure 11: F1 against training-data volume at two tolerances, single-split protocol. The ±3 curve is flat for the strong model (0.676 to 0.694, overlapping intervals) while the exact-match curve rises monotonically (0.455 to 0.505), though its endpoint intervals still overlap slightly. Finding a peptide saturates with data before placing its ends does.

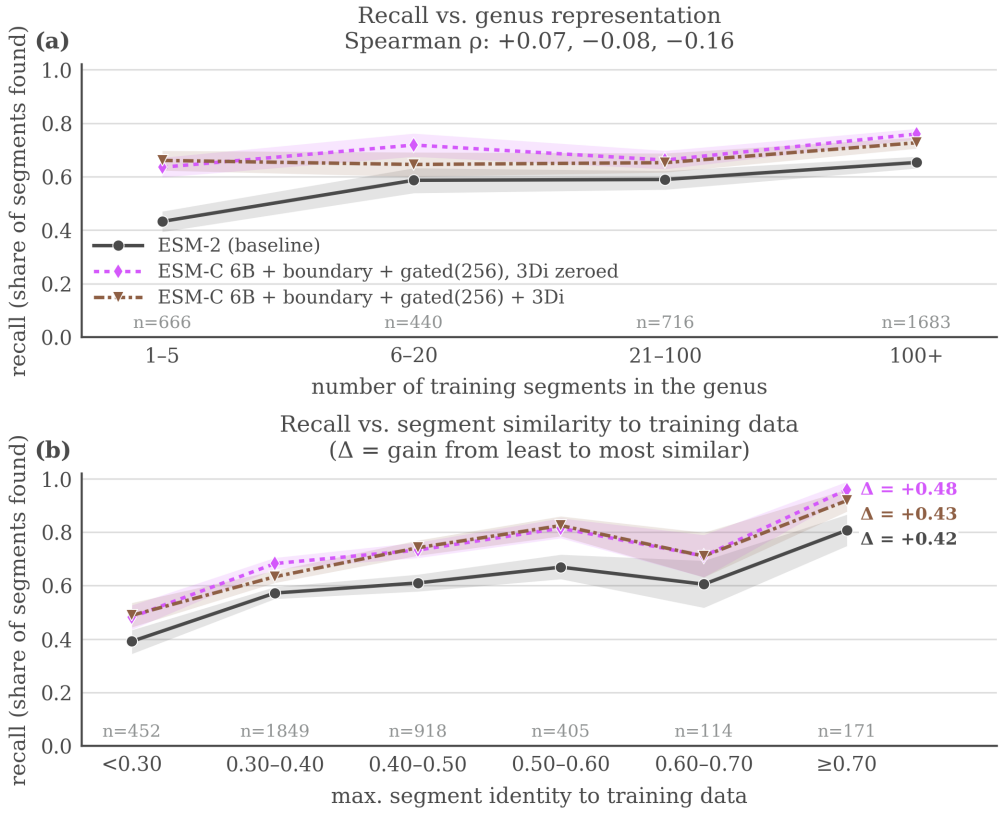

Figure 12: Recall on held-out peptides as a function of (a) how well-represented the protein's genus is in training, and (b) maximum sequence identity to a training segment, single-split protocol. For the two ESM-C models recall tracks maximum identity to a training segment far more than genus abundance; for the ESM-2 baseline the two axes are closer to comparable. The x axis in (a) counts training *segments* in the genus, not proteins.